

Reliability Under Deployment Shift

Distribution Shift, OOD Detection, Adversarial Robustness, and Conformal Prediction

Graduate Deep Learning Lecture

One deployment question

Core setup

$$P_s(X, Y) \neq P_t(X, Y).$$

- P_s : *source* or training distribution; P_t : *target* or deployment distribution.
- Interpretation: the training world and the deployment world are not the same.
- Why it matters: low source error does *not* automatically imply low deployment error.

The lecture question

When the world changes at test time, should we **reweight the loss**, **reject the input**, **defend against worst-case perturbations**, or **return a larger uncertainty set**?

Running example: the shelter classifier

A classifier is trained on clean studio photos of

{cat, dog, rabbit}.

At deployment, an animal shelter sees:

- darker photos and cluttered backgrounds;
- many more dogs than rabbits;
- new animals such as hamsters or foxes;
- possible adversarial stickers or pixel perturbations;
- a request for reliable set-valued predictions.

The same running example is reused across the deck so the four frameworks look like responses to one deployment story.

Mismatch	Best first response
Lighting / background change	Covariate-shift correction
Class-prior change	Label-shift correction
New animal appears	OOD detection / abstention
Adversarial perturbation	Adversarial robustness
Need safe prediction set	Conformal prediction

One factorization organizes most of the taxonomy

Factorization anchor

$$p(x, y) = p(y \mid x)p(x) = p(x \mid y)p(y).$$

- $p(x)$: how inputs appear; $p(y)$: class frequencies.
- $p(y \mid x)$: the correct label rule for an input.
- $p(x \mid y)$: how each class looks internally.
- Interpretation: to diagnose shift, ask *which factor changed*.
- Why it matters: different changes call for different corrections.

Taxonomy: same classes, new classes, worst-case perturbations

Case	What changes?	Stable quantity or symptom	Typical response
Covariate shift	$p_t(x) \neq p_s(x)$	same label rule $p_t(y x) = p_s(y x)$	Reweight examples
Label shift	$p_t(y) \neq p_s(y)$	same class appearance $p_t(x y) = p_s(x y)$	Reweight class posteriors
Semantic shift / out-of-distribution (OOD)	label set no longer complete	new classes or changed semantics appear	Score and reject
Adversarial shift	attacker perturbs the input	worst-case loss matters inside a threat model	Min-max train/evaluate
Uncertainty-set need	one label is too brittle	formal coverage guarantee is desired	Conformal prediction

Common beginner mistake

A corrupted dog image is usually *shifted ID*, not semantic OOD. A fox image is semantic OOD.

Micro-prerequisites and local notation

Idea / symbol	Equation or phrase	How it will be used later
Bayes' rule	$p(y x) = p(x y)p(y)/p(x)$	changed priors tilt posteriors under label shift
Risk $R_P(f)$	$\mathbb{E}_{(X,Y) \sim P}[\ell(f(X), Y)]$	target risk, weighted risk, and robust risk
Gradient	$g(x + \delta) \approx g(x) + \nabla g(x)^\top \delta$	local attack directions and optimization intuition
Budget ε	$\ \delta\ _p \leq \varepsilon$	defines the adversarial threat set
Conformal quantile $\hat{q}$	calibrated score threshold	decides which labels enter a prediction set
Core symbols	$w(x), r(y), S(x), \tau, V(x, y)$	reweighting, OOD scores, thresholds, and set construction

- Self-study cue: later formulas mostly do one of three things — reweight, threshold, or optimize against a worst case.

Near-OOD, far-OOD, and benchmark traps

- **Near-OOD**: semantically outside the label set but visually similar to ID. Example: fox versus dog.
- **Far-OOD**: semantically and visually far away. Example: airplane versus pet photo.
- Near-OOD is usually harder because representations overlap more strongly.

Why OOD evaluation is fragile

Preprocessing mismatch, background shortcuts, ID/OOD contamination, mixed shift types, easy far-OOD sets, and tuning thresholds on the test OOD set can all distort conclusions.

- Bottom line: benchmark papers can change practice as much as algorithm papers.

Target risk is the quantity we actually care about

Deployment objective

$$R_t(f) = \mathbb{E}_{(X,Y) \sim P_t}[\ell(f(X), Y)].$$

- f : predictor; $\ell(f(x), y)$: per-example loss.
- Interpretation: deployment performance is an expectation under the *target* world, not the source world.
- Why it matters: shift methods try to rewrite or approximate $R_t(f)$ using the data we actually have.

Two classical corrections

Under **covariate shift**, reweight examples. Under **label shift**, reweight classes.

Covariate shift: the task is stable, the input mix changes

Assumption

$$p_t(x) \neq p_s(x), \quad p_t(y | x) = p_s(y | x).$$

- Interpretation: dark dog photos may be more common at deployment, but the correct label for a given image has not changed.
- In the shelter example, camera, pose, and background distributions shift.
- This is the setting where source labels remain useful *if we count source examples differently*.

Covariate shift: where importance weighting comes from

Start from the target risk

$$R_t(f) = \int \ell(f(x), y) p_t(x, y) dx dy.$$

$$\begin{aligned} R_t(f) &= \int \ell(f(x), y) p_t(x) p_t(y | x) dx dy \\ &= \int \ell(f(x), y) p_t(x) p_s(y | x) dx dy \\ &= \int \ell(f(x), y) \frac{p_t(x)}{p_s(x)} p_s(x, y) dx dy \\ &= \mathbb{E}_{P_s}[w(X) \ell(f(X), Y)]. \end{aligned}$$

- $w(x) = p_t(x)/p_s(x)$ is the **importance weight**.
- Interpretation: source examples count in proportion to how representative they are of deployment.
- Why it matters: the whole covariate-shift program is this single multiply-and-divide move.

Importance weighting in practice

$$\widehat{R}_t(f) = \frac{1}{n} \sum_{i=1}^n \widehat{w}(x_i) \ell(f(x_i), y_i).$$

Domain-classifier recipe

Label target examples $D = 1$, source examples $D = 0$, and train $\hat{\eta}(x) \approx P(D = 1 \mid x)$. Bayes' rule gives

$$\frac{p_t(x)}{p_s(x)} = \frac{P(D = 1 \mid x) P(D = 0)}{P(D = 0 \mid x) P(D = 1)}.$$

Therefore estimate

$$\widehat{w}(x) \approx \frac{\hat{\eta}(x)}{1 - \hat{\eta}(x)} \frac{\hat{\pi}_s}{\hat{\pi}_t},$$

where $\hat{\pi}_s, \hat{\pi}_t$ are the source/target proportions used while training the domain classifier.

With balanced source-target batches, $\hat{\pi}_s = \hat{\pi}_t$, so

$$\widehat{w}(x) \approx \hat{\eta}(x)/(1 - \hat{\eta}(x)).$$

- Direct density-ratio methods estimate $p_t(x)/p_s(x)$ without modeling both densities separately.
- Clip or normalize large weights to reduce variance.
- Use the weighted loss to *evaluate* or *retrain*.
- Caveat: if $p_s(x) = 0$ where $p_t(x) > 0$, reweighting cannot invent labels there.

Label shift: class frequencies change, class appearance stays stable

Assumption

$$p_t(y) \neq p_s(y), \quad p_t(x | y) = p_s(x | y).$$

- Interpretation: dogs become more common than rabbits, but dogs still look like dogs within their class.
- Bayes' rule says changed priors $p(y)$ must change posteriors $p(y | x)$.
- This is a *class reweighting* problem, not an *example reweighting* problem.

Label shift: Bayes' rule gives posterior correction

Assumption

$$p_t(x | y) = p_s(x | y), \quad p_t(y) \neq p_s(y).$$

$$\begin{aligned} p_t(y | x) &= \frac{p_t(x | y) p_t(y)}{p_t(x)} \propto p_s(x | y) p_t(y) \\ &= \frac{p_s(y | x) p_s(x)}{p_s(y)} p_t(y) \propto \frac{p_t(y)}{p_s(y)} p_s(y | x). \end{aligned}$$

Define the **prior ratio**

$$r(y) = \frac{p_t(y)}{p_s(y)} \quad \implies \quad p_t(y | x) = \frac{r(y) p_s(y | x)}{\sum_{y'} r(y') p_s(y' | x)}.$$

- Interpretation: keep the source evidence $p_s(y | x)$, tilt it by changed class prevalence, then renormalize.
- Why it matters: class-prior shift is a class-weighting problem, not an example-weighting problem.

Numeric label-shift example

Define u_y as the **unnormalized corrected score** for class y :

$$u_y = r(y) p_s(y \mid x), \quad p_t(y \mid x) = \frac{u_y}{Z}, \quad Z = \sum_{y'} u_{y'}.$$

Class y	Source posterior $p_s(y \mid x)$	Prior ratio $r(y)$	Unnormalized score u_y
cat	0.40	1	$1 \cdot 0.40 = 0.40$
dog	0.35	2	$2 \cdot 0.35 = 0.70$
rabbit	0.25	0.5	$0.5 \cdot 0.25 = 0.125$

$$Z = 0.40 + 0.70 + 0.125 = 1.225, \quad p_t(\text{cat}, \text{dog}, \text{rabbit} \mid x) \approx (0.327, 0.571, 0.102).$$

- “Unnormalized” means the scores are not probabilities yet: they sum to 1.225, not 1.
- Dog becomes dominant because its deployment prior ratio boosts its source evidence.

Black-box shift estimation (BBSE)

Notation

- **BBSE**: estimate target class priors using only a trained classifier's predictions and source validation behavior.
- $\hat{Y} = f(X)$: classifier prediction; $\hat{y}$ is a possible predicted class.
- $\pi_t(y) = P_t(Y = y)$, $\pi_s(y) = P_s(Y = y)$: target/source class priors.
- $C_{\hat{y},y} = P_s(\hat{Y} = \hat{y} \mid Y = y)$: source confusion matrix estimated from source validation predictions.

From predictions to true-class priors

$$q_t(\hat{y}) = P_t(\hat{Y} = \hat{y})$$

$$q_t(\hat{y}) = \sum_y P_t(\hat{Y} = \hat{y} \mid Y = y) \pi_t(y).$$

Use the label-shift assumption

Under label shift, the classifier's class-conditional prediction behavior is stable:

$$P_t(\hat{Y} = \hat{y} \mid Y = y) = P_s(\hat{Y} = \hat{y} \mid Y = y) = C_{\hat{y},y}.$$

$$q_t(\hat{y}) = \sum_y C_{\hat{y},y} \pi_t(y), \quad q_t = C \pi_t.$$

$$\pi_t = C^{-1} q_t \quad \text{if } C \text{ is invertible.}$$

- q_t is observed from target predicted-label counts.
- Solve for π_t , then use $r(y) = \pi_t(y)/\pi_s(y)$ in posterior correction.

BBSE numeric example: target priors from predictions

What we observe

Two classes: cat and dog. From source validation data,

$$C = \begin{pmatrix} 0.8 & 0.2 \\ 0.2 & 0.8 \end{pmatrix}, \quad q_t = \begin{pmatrix} 0.56 \\ 0.44 \end{pmatrix}.$$

- $C_{\hat{y},y} = P_s(\hat{Y} = \hat{y} \mid Y = y)$: source confusion matrix; rows are predicted labels, columns are true labels.
- Count formula:
 $\hat{C}_{\hat{y},y} = \#\{i : y_i = y, f(x_i) = \hat{y}\} / \#\{i : y_i = y\}$.
- $q_t(\hat{y}) = P_t(\hat{Y} = \hat{y})$: observed target predicted-label distribution.
- $\pi_t(y) = P_t(Y = y)$: unknown target true-class prior.

Infer hidden true priors

Let

$$\pi_t = \begin{pmatrix} \pi_{\text{cat}} \\ \pi_{\text{dog}} \end{pmatrix}, \quad \pi_{\text{cat}} + \pi_{\text{dog}} = 1.$$

BBSE uses $q_t = C\pi_t$:

$$0.56 = 0.8\pi_{\text{cat}} + 0.2\pi_{\text{dog}} = 0.8\pi_{\text{cat}} + 0.2(1 - \pi_{\text{cat}}).$$

$$0.56 = 0.6\pi_{\text{cat}} + 0.2 \Rightarrow \pi_{\text{cat}} = 0.60, \quad \pi_{\text{dog}} = 0.40.$$

- Predicted-label counts plus the known error pattern reveal target class frequencies.

Covariate shift versus label shift

Setting	Stable quantity	Main correction	Main failure mode
Covariate shift	$p_t(y \mid x) = p_s(y \mid x)$	example weights $w(x) = p_t(x)/p_s(x)$	poor support overlap or wrong assumption
Label shift	$p_t(x \mid y) = p_s(x \mid y)$	prior ratios $r(y) = p_t(y)/p_s(y)$	unstable prior estimates or ill-conditioned confusion matrix

- Both corrections keep the original label set.
- Neither one solves semantic novelty: if a fox appears, OOD detection is the right next tool.

OOD detection is mostly score plus threshold

Ideal but unavailable test

$$\Lambda(x) = \frac{p_{\text{ID}}(x)}{p_{\text{OOD}}(x)}.$$

- ID means *in-distribution*; OOD means *out-of-distribution*.
- Interpretation: if both ID and OOD densities were known, threshold a likelihood ratio.
- Practical problem: p_{OOD} is usually broad, unknown, and heterogeneous.
- Practical response: choose a surrogate score $S(x)$ that is larger on ID points, then accept if $S(x) \geq \tau$.

Key distinction

OOD detection asks whether the input should have been forced into the training label set in the first place.

Family 1: confidence scores — MSP

Maximum softmax probability (MSP)

$$S_{\text{MSP}}(x) = \max_y p_{\theta}(y \mid x), \quad p_{\theta}(y \mid x) = \frac{e^{z_y(x)}}{\sum_k e^{z_k(x)}}.$$

- $z_y(x)$ are logits; MSP is the largest predicted class probability.
- Interpretation: highly peaked class belief is treated as more ID-like.
- Strength: extremely simple and historically strong as a baseline.
- Limitation: MSP only sees *relative* probabilities and can still be overconfident on unfamiliar inputs.

Family 2: logit-scale scores — energy

Energy score

$$S_{\text{energy}}(x) = \log \sum_{y=1}^K \exp(z_y(x)).$$

- Larger score means more ID-like in this lecture.
- Interpretation: log-sum-exp summarizes the *whole* logit vector, not just the winning class.
- Why it matters: energy can exploit absolute logit scale, which MSP discards.

Useful identity

$$\max_y z_y \leq \log \sum_y e^{z_y} \leq \max_y z_y + \log K.$$

Energy tracks overall logit level while still aggregating across classes.

Why MSP and energy can disagree

Suppose we add the same constant c to every logit.

MSP is unchanged

Softmax probabilities do not change, so $S_{\text{MSP}}(x)$ is identical.

Energy shifts by the constant

$$\log \sum_y e^{z_y + c} = c + \log \sum_y e^{z_y}.$$

Case 1: sharp posterior

One logit dominates:

$$z_a \gg z_b \quad (b \neq a), \quad \log \sum_y e^{z_y} \approx z_a.$$

Energy is almost the winning logit.

Case 2: near-tied posterior

All K logits are near level a :

$$z_y \approx a \quad \Rightarrow \quad \log \sum_y e^{z_y} \approx a + \log K.$$

Energy includes shared logit level plus a $\log K$ term.

Family 3: feature-density scores — Mahalanobis

Gaussian feature model

$$\phi(x) \mid Y = y \sim \mathcal{N}(\mu_y, \Sigma) \implies S_{\text{Mah}}(x) = -\min_y (\phi(x) - \mu_y)^\top \Sigma^{-1} (\phi(x) - \mu_y).$$

Estimate from labeled source features

Let $z_i = \phi(x_i)$, $n_y = \#\{i : y_i = y\}$, $N = \sum_y n_y$, and K be the number of classes.

$$\hat{\mu}_y = \frac{1}{n_y} \sum_{i:y_i=y} z_i, \quad \hat{\Sigma} = \frac{1}{N - K} \sum_y \sum_{i:y_i=y} (z_i - \hat{\mu}_y)(z_i - \hat{\mu}_y)^\top.$$

- μ_y is the class center in feature space; Σ is a pooled within-class covariance shared by all classes.
- Interpretation: ID examples should lie near at least one class cluster after covariance-aware rescaling.
- Why it matters: distances in stable feature directions count more than distances in highly variable directions.

Two historical bridges: sharpen the score or change training

ODIN: test-time score sharpening

Goal: make confidence separate ID from OOD better before thresholding.

$$p_T(y | x) = \frac{e^{z_y(x)/T}}{\sum_k e^{z_k(x)/T}}, \quad \hat{y} = \operatorname{argmax}_y p_T(y | x),$$

$$x' = x + \varepsilon \operatorname{sign}(\nabla_x \log p_T(\hat{y} | x)),$$

$$S_{\text{ODIN}}(x) = \max_y p_T(y | x').$$

- Let $s(x) = \log p_T(\hat{y} | x)$. First order:
 $s(x') - s(x) \approx \varepsilon \|\nabla_x s(x)\|_1$.
- ID inputs are near learned class regions, so the nudge reinforces real class evidence.
- OOD predictions are often accidental, so the same nudge usually boosts confidence less.

Outlier Exposure and OpenMax

OE changes training rather than only the test-time score:

$$\mathcal{L}_{\text{OE}} = \mathcal{L}_{\text{cls}} + \lambda \mathcal{H}(u, p_\theta(\cdot | x_{\text{out}})).$$

- u : uniform class distribution; $\mathcal{H}$: cross-entropy.
- OE teaches the classifier to stay low-confidence on auxiliary outliers.
- OpenMax reallocates some mass to an explicit “unknown” class.

Choosing and evaluating the OOD threshold

Three separate choices

- 1 score $S(x)$: MSP, energy, Mahalanobis, ...
- 2 threshold τ : the accept/reject cutoff;
- 3 calibration protocol: which validation data are allowed to tune τ .

Decision convention

$S(x) \geq \tau \Rightarrow$ accept as ID, $S(x) < \tau \Rightarrow$ reject as OOD.

- Example: set τ so that 95% of ID validation inputs are accepted.
- Do not tune τ on the test OOD benchmark.

Common metrics

- **AUROC**: area under ROC curve; ROC = receiver operating characteristic. Measures ranking quality.
- **AUPR**: area under precision–recall curve; sensitive to class imbalance.
- **FPR95**: false positive rate at 95% TPR; TPR = true positive rate. OOD wrongly accepted as ID.
- AUROC/AUPR summarize many thresholds; FPR95 checks one deployment-like operating point.

Recent OOD work: what actually changed?

- **Benchmark repair:** NINCO, ImageNet-OOD, and OpenOOD made evaluation harder and cleaner, especially for near-OOD cases.
- **Learned negatives:** OE, NPOS, and SAL changed training by supplying auxiliary or synthesized non-ID examples.
- **Stronger representations:** vision-language model (VLM)- and Contrastive Language-Image Pretraining (CLIP)-style pipelines kept the same score-threshold skeleton but changed the feature space and the scale of pretraining.

Organizing principle

Cluster papers by *what changed in the pipeline*: the benchmark, the negative data, or the representation.

OOD section recap

- Semantic OOD is not the same as ordinary covariate shift.
- Most OOD methods fit one skeleton:

choose $S(x)$ $\rightarrow$ choose τ $\rightarrow$ accept or reject.

- The main score families are MSP, energy, and feature-density scores.
- Benchmark realism is a first-class scientific issue, not a peripheral detail.

Threat model first, always

Threat model questions

What can the attacker modify? How large can the modification be? Does the attacker know the model? Is the attack targeted or untargeted?

- Without a threat model, the phrase “robust model” is incomplete.
- Common constraint:

$$\Delta(x) = \{\delta : \|\delta\|_p \leq \varepsilon, x + \delta \in [0, 1]^d\}.$$

- Interpretation: robustness is defined relative to an allowed perturbation set.

Robust loss and robust risk

Replace average-case loss by worst-case loss

$$\ell_{\text{rob}}(f; x, y) = \max_{\delta \in \Delta(x)} \ell(f(x + \delta), y), \quad R_{\text{rob}}(f) = \mathbb{E}[\ell_{\text{rob}}(f; X, Y)].$$

- ℓ_{rob} is the largest loss within the allowed perturbation set.
- Interpretation: evaluate the model on the *hardest nearby example*, not only the clean one.
- Why it matters: the learning problem becomes min-max optimization.

Training objective

$$\min_{\theta} \mathbb{E} \left[\max_{\delta \in \Delta(X)} \ell(f_{\theta}(X + \delta), Y) \right].$$

FGSM: why the sign of the gradient appears

Linearize by first-order Taylor

Define $L(x) = \ell(f_\theta(x), y)$ and $g = \nabla_x L(x)$. For small δ ,

$$L(x + \delta) = L(x) + g^\top \delta + O(\|\delta\|^2) \approx L(x) + g^\top \delta.$$

Assume: L is differentiable near x , ε is small, and higher-order terms are ignored.

$$\max_{|\delta_i| \leq \varepsilon} L(x + \delta) \approx \max_{|\delta_i| \leq \varepsilon} g^\top \delta.$$

Separate by coordinates

$$g^\top \delta = \sum_i g_i \delta_i,$$

$$\max_{|\delta_i| \leq \varepsilon} g^\top \delta = \sum_i \max_{|\delta_i| \leq \varepsilon} g_i \delta_i.$$

Solve one coordinate

Each coordinate is independent:

$$\delta_i^* = \begin{cases} \varepsilon, & g_i > 0, \\ -\varepsilon, & g_i < 0, \\ 0, & g_i = 0 \end{cases} = \varepsilon \operatorname{sign}(g_i).$$

Exact optimizer

$$\delta_{\text{FGSM}} = \varepsilon \operatorname{sign}(g) = \varepsilon \operatorname{sign}(\nabla_x \ell(f_\theta(x), y)).$$

- FGSM means **fast gradient sign method**.
- Each coordinate moves as far as allowed in the local loss-increasing direction.

Projected gradient descent (PGD)

What does the constraint mean?

$$\|\delta\|_p = \left(\sum_j |\delta_j|^p \right)^{1/p}, \quad \|\delta\|_\infty = \max_j |\delta_j|.$$

- δ : perturbation added to the input x .
- $\|\cdot\|_p$: a way to measure perturbation size.
- ℓ_∞ budget $\|\delta\|_\infty \leq \varepsilon$: every coordinate changes by at most ε .
- Π : projection back into the allowed perturbation set.

Interpretation: PGD means projected gradient descent: many small adversarial steps, each snapped back into the allowed set.

Algorithm: common ℓ_∞ PGD attack

- 1 Initialize $\delta^0 = 0$ or random with $\|\delta^0\|_\infty \leq \varepsilon$.
- 2 For $t = 0, \dots, K - 1$, compute

$$g^t = \nabla_\delta \ell(f_\theta(x + \delta^t), y).$$

- 3 Take a loss-increasing step and project:

$$\delta^{t+1} = \Pi_{\|\delta\|_\infty \leq \varepsilon} \left(\delta^t + \alpha \operatorname{sign}(g^t) \right).$$

- 4 Return the adversarial input $x_{\text{adv}} = x + \delta^K$.

Adversarial training: optimize on attacked examples

Canonical procedure

For each minibatch, approximately solve the inner attack problem, create adversarial examples $x_i + \hat{\delta}(x_i)$, then update θ using their loss.

- Strength: strongest widely used empirical defense baseline.
- Limitation: expensive because each gradient step contains an inner attack loop.
- Practical meaning: the model learns against the perturbations it is evaluated on.

Clean-vs-robust tension

Features that help clean accuracy can be exactly the ones an adversary perturbs most easily.

TRADES: explicitly stabilize the predictive distribution

TRADES objective

$$\mathcal{L}_{\text{TRADES}}(x, y) = -\log p_{\theta}(y \mid x) + \beta \max_{\|\delta\|_p \leq \epsilon} \text{KL}(p_{\theta}(\cdot \mid x) \parallel p_{\theta}(\cdot \mid x + \delta)).$$

- KL is the **Kullback–Leibler divergence**.
- TRADES is a trade-off objective: the first term keeps clean accuracy high, the second penalizes prediction change inside the perturbation ball.
- β controls the clean-versus-robust balance.

Interval bound propagation (IBP): interval arithmetic through the network

Goal and invariant

Certify all $x' \in [x - \varepsilon, x + \varepsilon]$ without checking them one by one:

$$\mathcal{R}_\ell = \{h_\ell(x') : x' \in [x - \varepsilon, x + \varepsilon]\} \subseteq [\underline{h}_\ell, \bar{h}_\ell].$$

IBP propagates this enclosing box layer by layer.

Affine layer: exact over a box

For $h = Wx + b$, split $W = W^+ + W^-$. Then

$$\underline{h} = W^+ \underline{x} + W^- \bar{x} + b,$$

$$\bar{h} = W^+ \bar{x} + W^- \underline{x} + b.$$

Positive weights preserve endpoints; negative weights flip them.

Nonlinear layers

For monotone activations, push bounds coordinatewise.
For ReLU:

$$h \in [\underline{h}, \bar{h}] \Rightarrow \text{ReLU}(h) \in [\text{ReLU}(\underline{h}), \text{ReLU}(\bar{h})].$$

Sound but loose

true reachable set $\subseteq$ IBP interval box.

If the IBP box is certified safe, the real network is safe.
The cost is looseness: intervals forget correlations between neurons.

IBP: certify using output-logit intervals

After all layers

For each class c , IBP gives a guaranteed final-logit interval:

$$z_c(x') \in [\underline{z}_c, \bar{z}_c] \quad \text{for every } x' \in [x - \varepsilon, x + \varepsilon].$$

Compare true class to each rival

Prediction stays y if $z_y(x') > z_c(x')$ for all $c \neq y$. Since

$$z_y(x') - z_c(x') \geq \underline{z}_y - \bar{z}_c,$$

it is enough to lower-bound every true-vs-rival margin.

Certificate condition

If the lowest possible true-class logit beats the highest possible rival logit,

$$\underline{z}_y > \max_{c \neq y} \bar{z}_c,$$

then no perturbation inside the input box can change the predicted label.

Robustness claims live or die by evaluation

- **Report the threat model:** norm, budget, attack steps, step size, restarts, and clean *and* robust accuracy.
- **Use strong attacks:** suites such as AutoAttack reduce weak-attack illusions and help expose gradient masking.
- **Test beyond one benchmark:** OODRobustBench showed robustness can collapse under distribution shift; AdvXL and Robust CLIP show the same min-max questions at foundation-model scale.

Bottom line

A defense should be judged by the strongest credible attack and by evaluation settings that match deployment, not by one easy benchmark.

Adversarial section recap

- Robustness is defined relative to a *threat model*.
- Canonical skeleton:

$$\text{define } \Delta(x) \rightarrow \max_{\delta \in \Delta(x)} \text{loss} \rightarrow \min_{\theta} \text{worst-case risk.}$$

- FGSM is the one-step linearized attack; PGD is the multi-step workhorse.
- TRADES manages the clean/robust tension; certification asks a different question from attack-based evaluation.

Conformal prediction returns a set, not just one label

Target guarantee

$$\mathbb{P}\{Y_{n+1} \in C(X_{n+1})\} \geq 1 - \alpha.$$

- $C(x)$ is a prediction set; α is the target miscoverage level.
- Interpretation: the true label should land inside the returned set at least $1 - \alpha$ of the time.
- Why it matters: when a single label is too brittle, the model can safely say “dog or rabbit.”

Name intuition

In English, **conformal** means fitting, matching, or being in agreement with something. For classification, a candidate label is kept when its score looks sufficiently *conforming* to calibration examples; the output is a set of labels that fits the desired coverage level.

Split conformal: score, calibrate, include

1. Score: how incompatible is a label?

Choose a nonconformity score $V(x, y)$, small when label y looks plausible for input x .

$V(x, y) = 1 - \hat{p}_y(x)$ is a common example.

On calibration examples, score only the true labels:

$$s_i = V(X_i, Y_i), \quad i = 1, \dots, m.$$

2. Calibrate: choose the threshold

Let m be the calibration-set size and let

$$r = \lceil (m + 1)(1 - \alpha) \rceil.$$

Set $\hat{q}$ to the r -th smallest value among $s_1, \dots, s_m$.

3. Include: return a label set

For a new input x , include every class label j whose score is below the calibrated threshold:

$$C(x) = \{j : V(x, j) \leq \hat{q}\}.$$

label	cat	dog	fox	$\hat{q} = 0.40$
$V(x, \text{label})$	0.80	0.20	0.35	

Then $C(x) = \{\text{dog}, \text{fox}\}$.

- Calibration tells us how large the *true label's* score usually gets.
- Here r is a rank in the sorted calibration scores; j indexes class labels.

Worked example: a full split-conformal set

Take $n = 9$ calibration scores for the true labels:

0.10, 0.15, 0.20, 0.30, 0.35, 0.40, 0.55, 0.70, 0.80.

For $\alpha = 0.2$,

$$k = \lceil (9 + 1)(0.8) \rceil = 8, \quad \hat{q} = 0.70.$$

Suppose a test image has class probabilities

$$\hat{p}(\text{dog} \mid x) = 0.45, \quad \hat{p}(\text{cat} \mid x) = 0.35, \quad \hat{p}(\text{rabbit} \mid x) = 0.20,$$

with score $V(x, y) = 1 - \hat{p}(y \mid x)$. Then the candidate-label scores are

0.55, 0.65, 0.80.

- Include labels with score at most 0.70.
- Result: $C(x) = \{\text{dog}, \text{cat}\}$.

APS and RAPS: better set scores for classification

Adaptive prediction sets (APS)

If labels are sorted by predicted probability and $r_y(x)$ is the rank of label y ,

$$V_{\text{APS}}(x, y) = \sum_{j=1}^{r_y(x)} \hat{p}_{(j)}(x).$$

- Interpretation: how much cumulative probability mass must be collected before reaching label y ?
- Regularized adaptive prediction sets (RAPS) add a rank penalty

$$V_{\text{RAPS}}(x, y) = V_{\text{APS}}(x, y) + \lambda (r_y(x) - k_{\text{reg}})_+,$$

to discourage very deep sets.

- Why it matters: APS/RAPS often produce smaller, better-shaped classification sets than the raw $1 - \hat{p}(y \mid x)$ score.

What to report for conformal methods

- score $V(x, y)$ being calibrated;
- calibration split and calibration size;
- target miscoverage level α ;
- empirical coverage and average set size (or interval length);
- subgroup- or shift-stratified coverage when relevant;
- adaptation protocol under shift: weighted, label-shift, ACI, PID, ECI, unlabeled-target, multi-source, ...

Do not over-interpret nominal coverage

A method can hit the target coverage level while producing sets that are too large to be operationally useful.

Conformal section recap

- Canonical skeleton:

choose score $\rightarrow$ calibrate quantile $\rightarrow$ return prediction set.

- Exchangeability gives exact split-conformal coverage; shift-aware methods change who counts or how the threshold moves.
- APS/RAPS improve classification-set efficiency; weighted and adaptive methods address deployment shift.
- The central trade-off is **coverage versus set size**, not just coverage alone.

Deployment decision matrix

Deployment cue	First response
The label set is incomplete at deployment	OOD detection / abstention
The semantic task itself changed	simple shift correction is probably invalid; reconsider the task formulation
An attacker can choose perturbations	adversarial robustness
A single prediction is too brittle	conformal prediction
The task is the same, but input or class proportions changed	covariate-shift or label-shift correction

Common failure modes across the whole lecture

Failure mode	What it looks like in practice
Wrong framing	Treating new semantic classes as if they were only a class-prior change; treating adversarial perturbations as ordinary covariate shift
Theory–data mismatch	Assuming covariate shift when $p(y \mid x)$ changed; assuming exchangeability under strong drift with no adaptation
Benchmark shortcuts	A detector wins by exploiting background or preprocessing artifacts rather than the intended phenomenon
Weak evaluation	A defense looks robust only because the attack is weak; a conformal method is assessed on the same data used for calibration
Single-metric thinking	High AUROC with poor thresholded behavior; nominal coverage with unusably large sets

Open problems that remain hard

- **Distribution shift:** detect when covariate- or label-shift assumptions are wrong; estimate target risk with weak support overlap.
- **OOD detection:** separate semantic novelty from benign covariate change; calibrate thresholds for heterogeneous OOD.
- **Adversarial robustness:** scale certified defenses and evaluate robustness jointly under natural and adversarial shift.
- **Conformal prediction:** move from marginal to useful conditional guarantees and adapt under strong nonstationarity.

Final synthesis: four canonical skeletons

Reweighting backbone

$$R_t(f) = \mathbb{E}_{P_s}[w(X)\ell(f(X), Y)]$$

$$p_t(y \mid x) \propto r(y)p_s(y \mid x)$$

Robustness backbone

$$\min_{\theta} \mathbb{E} \left[\max_{\delta \in \Delta(X)} \ell(f_{\theta}(X + \delta), Y) \right]$$

OOD backbone

$$\phi_{\tau}(x) = \mathbf{1}\{S(x) \geq \tau\}$$

Conformal backbone

$$C(x) = \{y : V(x, y) \leq \hat{q}\}$$

- Reweight, reject, defend, or return a set — the correct action depends on what changed.

Appendix: local symbol table

Symbol	Meaning	Plain-language interpretation
P_s, P_t	source and target distributions	training world and deployment world
X, Y	input and label	image and class name
f	predictor	deployed model
$\ell(f(x), y)$	loss	how wrong one prediction is
$R_P(f)$	risk under P	average loss on a distribution
$w(x)$	importance weight	target relevance of input x
$r(y)$	prior ratio	how much class y prevalence changed
$S(x)$	OOD score	scalar used for accept/reject
τ	threshold	cutoff for a decision
δ, ε	perturbation and budget	allowed adversarial change and its size
$V(x, y)$	nonconformity score	how incompatible label y looks
$\hat{q}$	calibrated threshold	score quantile for conformal prediction

Appendix: a tiny BBSE solve

Suppose the source confusion matrix is

$$C = \begin{pmatrix} 0.9 & 0.2 \\ 0.1 & 0.8 \end{pmatrix}, \quad q_t = \begin{pmatrix} 0.58 \\ 0.42 \end{pmatrix}.$$

Then solve

$$q_t = C\pi_t, \quad \pi_t = C^{-1}q_t.$$

Because

$$C^{-1} = \frac{1}{0.9 \cdot 0.8 - 0.2 \cdot 0.1} \begin{pmatrix} 0.8 & -0.2 \\ -0.1 & 0.9 \end{pmatrix},$$

we obtain approximately

$$\pi_t \approx \begin{pmatrix} 0.514 \\ 0.486 \end{pmatrix}.$$

Appendix: coordinate-wise FGSM derivation

$$\max_{\|\delta\|_\infty \leq \varepsilon} g^\top \delta = \max \sum_{i=1}^d g_i \delta_i.$$

Under $|\delta_i| \leq \varepsilon$, each coordinate is optimized independently:

$$\max_{|\delta_i| \leq \varepsilon} g_i \delta_i = \varepsilon |g_i|, \quad \delta_i^* = \varepsilon \operatorname{sign}(g_i).$$

Therefore

$$\delta^* = \varepsilon \operatorname{sign}(g).$$

- Interpretation: the sign is exact for the linearized ℓ_∞ subproblem.

Appendix: why the split-conformal guarantee appears

Main intuition

The calibration set tells us how large the *true label's* score usually gets. At test time, we include all labels whose scores are still below that calibrated cutoff.

- If calibration and test examples are exchangeable, the next true-label score behaves like one more calibration score.
- Therefore a high calibration quantile controls how often the true label's score lands above the threshold.
- The result is marginal coverage:

$$\mathbb{P}\{Y_{\text{test}} \in C(X_{\text{test}})\} \geq 1 - \alpha.$$

- The rank-symmetry proof is on the next appendix slide.

Important limitation

The guarantee is marginal, not conditional on every subgroup or every possible input.

Appendix: exchangeability and rank symmetry

Rank argument

If $V_1, \dots, V_n, V_{n+1}$ are exchangeable, then every ordering of these $n + 1$ scores is equally likely (up to ties). Hence the rank of V_{n+1} is uniform on $\{1, \dots, n + 1\}$.

- Choose the $k = \lceil (n + 1)(1 - \alpha) \rceil$ -th calibration quantile.
- Then the probability that the future score falls above that threshold is at most α .
- This is the engine behind split-conformal finite-sample coverage.

Appendix: split-conformal guarantee details

Failure is a rank event

For the test point, define the true-label score

$$s_{m+1} = V(X_{\text{test}}, Y_{\text{test}}).$$

Because $C(x) = \{y : V(x, y) \leq \hat{q}\}$,

$$Y_{\text{test}} \notin C(X_{\text{test}}) \iff s_{m+1} > \hat{q}.$$

Tiny example

Let $m = 9$ and $\alpha = 0.1$. After adding the test score, there are 10 scores, so

$$r = \lceil (9 + 1)(0.9) \rceil = 9.$$

Rank symmetry leaves probability at most 1/10 that the test score lands above the calibrated threshold, so

$$\mathbb{P}\{Y_{\text{test}} \in C(X_{\text{test}})\} \geq 0.9.$$

Appendix: weighted conformal under covariate shift

Reweight calibration scores by target relevance

$$w(x) = \frac{p_t(x)}{p_s(x)}, \quad \tilde{w}_i(x) = w(X_i), \quad \tilde{w}_{n+1}(x) = w(x).$$

Normalize the weights:

$$p_i^w(x) = \frac{\tilde{w}_i(x)}{\sum_{j=1}^{n+1} \tilde{w}_j(x)}.$$

Define the weighted threshold as the smallest t such that

$$\sum_{i=1}^n p_i^w(x) \mathbf{1}\{V_i \leq t\} \geq 1 - \alpha.$$

Call this threshold $\hat{q}_w(x)$, then output

$$C_w(x) = \{y : V(x, y) \leq \hat{q}_w(x)\}.$$

- $V_i = V(X_i, Y_i)$ are calibration scores; larger weights make a calibration point count more.
- Interpretation: calibration examples that look more like deployment contribute more to the quantile.
- Why the test-point weight appears: weighted conformal compares the future score against calibration scores *plus itself*, so the test point must also carry weight.

Appendix: shift-aware conformal families

Family	What changes?	Core idea and representative variants
Label-shift conformal	calibration weights depend on class	use $\omega_i = r(Y_i)$ so classes more common at deployment count more
Adaptive / online conformal	threshold dynamics change over time	ACI updates α_t or q_t from recent misses; proportional-integral-derivative (PID) control and error-quantified conformal inference (ECI) make the update smoother or more informative
Unlabeled-target adaptation	score or threshold uses target covariates	exploit target uncertainty statistics even without target labels
Multi-source conformal	calibration source is pooled and weighted	estimate the target quantile from several candidate source domains

Organizing principle

Modern conformal variants usually modify one ingredient: the **score**, the **calibration weights**, or the **threshold dynamics**.

Appendix: IBP propagation details

For an affine layer $h = Wx + b$, write $W = W^+ + W^-$ with

$$W_{ij}^+ = \max(W_{ij}, 0), \quad W_{ij}^- = \min(W_{ij}, 0).$$

Then interval bounds propagate as

$$\underline{h} = W^+ \underline{x} + W^- \bar{x} + b, \quad \bar{h} = W^+ \bar{x} + W^- \underline{x} + b.$$

For monotone nonlinearities such as ReLU,

$$\underline{z} = \text{ReLU}(\underline{h}), \quad \bar{z} = \text{ReLU}(\bar{h}).$$

If the true-class lower logit exceeds every other class upper logit, the prediction is certified constant inside the perturbation box.

Appendix: why source accuracy alone is not enough

Classical domain-adaptation bound

$$\epsilon_t(h) \leq \epsilon_s(h) + \frac{1}{2}d_{\mathcal{H}\Delta\mathcal{H}}(P_s, P_t) + \lambda^*.$$

- $\epsilon_s(h), \epsilon_t(h)$: source and target error of hypothesis h .
- $d_{\mathcal{H}\Delta\mathcal{H}}$: how distinguishable the domains are using the hypothesis class.
- λ^* : error of the best single model that can do well on both domains.
- Interpretation: low source error is not enough if the domains are far apart or if no shared good classifier exists.

Appendix: how to read the literature without one-paper-per-slide thinking

- **Distribution shift trunk:** reweight risk or correct priors; theory explains when source success can transfer.
- **OOD trunk:** score-threshold detectors, then better negative data and much better benchmarks.
- **Adversarial trunk:** attacks, min-max training, trade-off objectives, stronger evaluation, then scaling to modern models.
- **Conformal trunk:** exchangeability and rank arguments, better scores, shift-aware weighting, online adaptation, and multi-source extensions.

Practical reading rule

For each area, learn one root paper, one canonical backbone paper, and one benchmark-shaping or recent deployment paper.

Appendix: representative papers by area

Area	Representative papers mentioned in the source
Distribution shift	Shimodaira (2000), Sugiyama et al. (2007), Saerens et al. (2002), Lipton et al. (2018), Ben-David et al. (2010)
OOD detection	Hendrycks–Gimpel (MSP), Liang et al. (ODIN), Lee et al. (Mahalanobis), Liu et al. (energy), Hendrycks et al. (OE), Bitterwolf et al. (NINCO), Yang et al. (ImageNet-OOD), Zhang et al. (OpenOOD)
Adversarial robustness	Goodfellow et al. (FGSM), Madry et al. (min–max training), Zhang et al. (TRADES), Athalye et al. (obfuscated gradients), Croce–Hein (AutoAttack), Gowal et al. (IBP), Li et al. (OODRobustBench), Wang et al. (AdvXL)
Conformal prediction	Vovk–Gammerman–Shafer, Lei et al. (split conformal), Romano et al. (APS), Angelopoulos et al. (RAPS/PID/foundations), Tibshirani et al. (weighted conformal), Podkopaev–Ramdas (label shift), Gibbs–Candès (ACI/arbitrary shift), Kasa et al. (unlabeled target adaptation), Liu et al. (multi-source), Wu et al. (ECI)

Appendix: recent field-shaping works by what changed

Area	Representative works	What changed and why it mattered
OOD	NINCO, ImageNet-OOD, OpenOOD, OOD-X	Changed the evaluation distribution and protocol, forcing detectors to survive cleaner near/far/full-spectrum benchmarks rather than easy artifact-heavy settings
Adversarial robustness	OODRobustBench, AdvXL, Robust CLIP	Changed the evaluation target, the training scale, and the downstream model ecosystem while keeping the min-max backbone intact
Conformal prediction	Gentle Introduction / Theoretical Foundations, PID, ECP, multi-source, ECI	Changed pedagogy, threshold dynamics, unlabeled-target adaptation, and target-quantile estimation; made clear that many methods differ only in score, weights, or threshold updates

Appendix: five self-check questions

- ① Why does covariate shift lead to example weights $w(x)$ rather than class weights?
- ② Why is OOD detection usually written as score plus threshold instead of as ordinary classification?
- ③ Why does the sign of the gradient solve the linearized ℓ_∞ attack?
- ④ Why does exchangeability make the conformal rank argument work?
- ⑤ Why can a benchmark paper change conclusions as much as a new algorithm paper?
 - If a student can answer these in plain English first, the main story of the deck is in place.